

Claim-Faithful Publication of a Formalised Mathematics Corpus

A systems companion on status registries, verified source anchors,
and a self-testing release gate

Will Cook

July 2026

Abstract

A proof assistant checks exactly the propositions its kernel elaborated, at exactly the generality it elaborated them. Everything a project publishes about those propositions — papers, a README, orientation files, machine-readable packets — is a lossy projection, and the characteristic projection failure is silent strengthening: an implication reads as an equivalence, finite evidence reads as an unbounded theorem, an open problem quietly leaves the page. We describe the publication architecture of a Lean corpus of 12,496 declarations around two open Erdős problems, in which a single claim registry with a seven-status taxonomy binds every public claim to kernel-checked declarations at pinned source coordinates; the authored papers carry their formal anchors inside ordinary mathematical phrases; generated reader surfaces operate under enforced byte budgets; and one release gate of 5,207 checks fails whenever any surface overstates a logical status, loses an open-problem boundary, drifts from its source coordinates, or exceeds its budget. The gate also carries adversarial fixtures for its own checkers, so a detector that stops detecting is itself a release failure. In a mutation study at a checkpoint where every check passes, nine of ten deliberate corruptions spanning these failure classes were caught by the gate; the escape — a weakened boundary clause that no anchor group had enumerated — became a new anchor and a pinned rejection fixture. We also record three defects the same machinery caught during ordinary development. The mathematics of the corpus is one instance; the design problem — publishing formal work without overstating it — faces any project whose public surfaces restate machine-checked content.

1 Introduction

The Lean kernel accepts a proof term for a stated proposition. That acceptance is narrow and exact: it covers the elaborated statement, under the stated hypotheses, at the stated generality, and nothing else. A formal development of useful size, however, is not read through its kernel. It is read through publication surfaces: an expository paper, a README, a scope statement, orientation documents, navigation indices, and increasingly through bounded machine-readable packets consumed by software agents. Each surface restates the mathematics in a form the kernel never sees, and each restatement can strengthen a claim without anyone deciding to strengthen it.

The corpus studied here formalises results around Erdős problems 249 and 257, two open irrationality questions. Both problems remain open, and the corpus says so; its value depends on saying so in every projection. This creates an unusually sharp version of a general problem. A library of settled mathematics that misdescribes a theorem’s strength embarrasses its authors. A corpus organised around open problems that misdescribes a conditional reduction as a proof, or lets a verified finite range read as an unbounded quantifier, damages the one thing it exists to establish: an exact public boundary between what is proved and what is not.

This paper describes the architecture that keeps those projections honest, as implemented in the public repository that also ships the mathematical gateway exposition [1] and a specialist companion [2]. Its components are ordinary — a JSON registry, some authored T_EX, generated Markdown, and roughly 7,800 lines of dependency-free Python — and that ordinariness is part of the point. The claims of this paper are about the invariants, not the implementation effort:

1. *A recorded authority order* (Section 4). Proof authority, claim-status authority, methodological authority, authored exposition, and generated navigation are distinct layers, machine-recorded in that order, and no layer may strengthen a statement it inherits from a layer above. Each evidence class must record what it does *not* decide.
2. *Claim status as a typed machine object* (Section 5). Every public claim carries one of seven statuses with structural rules (an open or cited claim must not carry proof declarations; a formal claim must), every remaining open proposition is a typed record, and the registry also carries explicit *non-claims*. All 96 claims are partitioned exactly once into 20 contribution families, and the partition is checked.
3. *Anchored exposition* (Section 6). Paper prose links each stated result to a pinned commit, file, and line; the visible text is an ordinary mathematical phrase, the coordinates underneath are validated to within a three-line window, and the rendered narrative is forbidden from printing raw paths, hashes, or Lean identifiers. Every remaining open proposition must resolve to exactly one authored problem statement in a paper, so an open problem cannot leave the exposition unnoticed.
4. *Budgeted projections* (Section 7). Generated reader surfaces are rebuilt from the registry by owners, tied together by content digests, and held under explicit byte budgets, so first contact stays readable as the corpus grows.
5. *A self-testing release gate* (Sections 8 and 9). One command runs 5,207 checks in about 145 seconds and fails on status drift, boundary loss, coordinate rot, stale projections, budget overflow, or untrusted proof code. The gate additionally runs 38 negative fixtures against its own checkers. In a systematic mutation study, nine of ten deliberate corruptions were detected, and the one escape was converted into a new anchor and fixture; during ordinary development the same machinery caught three real defects, including an edit that deleted a load-bearing negative sentence.

Two boundaries on the claims of this paper should be stated at once, and are developed in Section 10. First, this is a case study of one corpus with one maintainer; no controlled comparison against conventional practice is attempted. Second, the gate checks the presence and consistency of statements the registry knows about; it does not judge the mathematical adequacy of unanchored prose. The architecture makes a specific class of publication defects mechanically detectable at commodity cost; it does not make exposition true.

On provenance: the development and its tooling were built by a single maintainer working with AI assistants; the Lean kernel, not any assistant, is the proof authority, and the release machinery described here exists partly *because* assistant-generated edits are a real source of drift. Two of the three development defects recorded in Section 9.2 were introduced by tooling-assisted editing passes and caught by the gate.

2 The published corpus

The artifact is a public Lean 4 package [3] building against Mathlib [4], together with its publication surfaces. At the checkpoint studied throughout this paper the corpus comprises 633 Lean modules and 12,496 declarations, of which 11,471 are theorem-like and 2,925 are

generated finite-certificate declarations. The registry curates 96 public claims in 20 contribution families, three typed remaining-open propositions, and six non-claims; the counts are stamped at the checkpoint and grow under development. The mathematical content — exact certificate equivalences, sharp denominator bounds, achievement-set geometry, and scoped countermodels for the two Erdős problems — is the subject of the gateway exposition [1] and the transport-curvature companion [2]; this paper does not restate it and claims no mathematical results.

The division of labour among the three papers is deliberate. The gateway owns the mathematics; the companion owns one specialist programme; this paper owns the question of how a corpus of that size is published without its projections outrunning its proofs. The mathematics serves here only as the live instance on which the machinery is exercised, and nothing below depends on the specific problems: any project whose public surfaces restate kernel-checked content faces the same failure classes.

3 The failure model

The architecture is organised against five concrete ways a truthful corpus drifts into an untruthful publication. All five were either observed in this project’s development or are one careless edit away; none requires bad faith.

Status inflation. A conditional reduction is summarised as a result; a one-way implication is remembered as the equivalence it almost is; a verified finite range is described with the unbounded quantifier it supports only conditionally. The opposite defect is quieter but also real: understatement, as when a proved equivalence is cautiously reported as a sufficient condition. Both directions misstate the mathematics; an exact status vocabulary must resist both.

Boundary erosion. The sentences that carry negative information — “this does not solve the problem”, “this bound does not supply the unbounded quantifier” — are precisely the ones a well-meaning copy edit deletes, because they read as redundant hedging to anyone who has not internalised why they are load-bearing.

Coordinate rot. Links from prose to source are correct on the day they are written. Source moves; line numbers shift; a renamed module orphans a hundred references. A reader who follows a stale anchor to the wrong declaration has been misinformed as surely as by a wrong sentence.

Projection fossilisation. Generated summaries, indices, and counts are rebuilt when someone remembers to rebuild them. A hand edit to a generated file, or a forgotten rebuild after an upstream change, leaves two public surfaces asserting different facts.

Attention overflow. First-contact surfaces grow monotonically under ordinary maintenance, until the document that was supposed to orient a reader in one minute no longer can. Unbounded growth is a truthfulness problem, not only a style problem: a boundary sentence that has scrolled out of the part people read might as well have been deleted.

4 Authority order

The registry records, as data, the order in which surfaces may be believed: first Lean source checked by the pinned kernel; second the claim registry (`docs/claims.json`) for release identity, status taxonomy, remaining open propositions, coordinates, and navigation relationships; third the methodology contract (`docs/methodology.json`); fourth the authored exposition; and last the README, scope, and navigation documents as reader projections. Two disciplines make this order load-bearing rather than decorative.

First, *downward-only strengthening*: a surface may compress or omit what it inherits, and may never strengthen it. The gate implements this directionally — every downstream surface is checked against the registry, and the registry is checked against the pinned Lean tree — so

an overstatement must appear upstream, in the one place designed to be reviewed line by line, or be caught.

Second, *explicit non-decision*. The methodology contract types its evidence classes (kernel check, repository check, human mathematical review), and each class must state both what it decides and what it does not decide; the contract validator rejects an evidence class missing either field. The same posture appears at the top of the order as a method axiom: kernel acceptance establishes the elaborated proposition, not that the proposition expresses the intended mathematics. Making every authority state its own boundary is the single design decision from which most of the machinery below follows.

The registry also records six explicit *non-claims*, three of which are provenance boundaries: the public corpus is a self-contained projection of a larger private research workflow, and the registry states as checkable data that no public claim rests on hidden proof bodies, private repositories, or any authority other than the pinned public sources. A first-contact test suite fails if any of these non-claims, or the plain-language sentences that carry them, disappear from the entry surfaces.

5 Claim status as a typed object

Each of the 96 public claims carries one status from a closed taxonomy of seven: *formalised here*, *proved here*, *conditional reduction*, *verified finite instance*, *unconditional progress*, *cited only*, and *open*. The taxonomy is deliberately finer than proved/unproved, because the corpus’s characteristic outputs — exact equivalences whose producers remain open, kernel-checked certificate ranges, denominator exclusions that bound without deciding — are exactly the results that coarse vocabularies inflate.

Statuses are structurally typed, not just enumerated. A claim whose status is *cited only* or *open* must not carry proof declarations; every other claim must carry at least one, and each declaration names a module and line at which the declaration is verified to exist, against the pinned formal-source commit rather than the working tree. Remaining open propositions are first-class records, and any claim asserting progress toward one must name its effect from a closed vocabulary — *proves*, *narrows*, *decomposes*, *re-expresses*, or *constrains* — so that “progress” cannot be claimed without saying what changed.

The methodology contract extends the same typing to *changes*. Eleven change classes (proof body changed, assumptions changed, finite certificate range changed, claim status changed, and so on) each carry minimum evidence requirements and forbidden effects. Two examples fix the flavour: a change of class *finite certificate range changed* is forbidden from carrying the effect *infer unbounded result*, and any change class that alters mathematical semantics is forbidden from downgrading human mathematical review to an automatic trigger. A rule in the contract must name the release guard that implements it, and the rendered `METHODOLOGY.md` must match its JSON source byte for byte.

Finally, the registry partitions all claims into 20 contribution families, each with a narrative owner, a status summary, a prior-art posture, and an open obligation or consumer. The gate enforces that the families cover every claim exactly once. The partition exists for editorial honesty: a claim that belongs to no family has no owner responsible for stating its boundary, and a claim in two families gets described twice, which is how wording divergence starts.

6 Anchored exposition

The authored papers link mathematics to source through macros whose visible output is an ordinary mathematical phrase and whose arguments carry the exact file, line, and declaration underneath. The papers pin a full commit hash; links resolve into that immutable tree, never into a floating branch. The release gate re-validates every link: the file must exist at the pinned

commit and the named declaration must appear within three lines of the stated coordinate. A separate rendering contract keeps the implementation layer out of the printed narrative: visible prose may not contain source paths, forty- character hashes, or raw Lean identifiers, and every source link must print readable words rather than an undifferentiated glyph.

Anchoring is bidirectional. Downward, each claim in the registry may name the paper label that presents it, and the gate fails if the label does not exist in the authored $\text{T}_{\text{E}}\text{X}$. Upward, an inventory derives every anchor in the papers — labelled sections and formal environments — and types each one as a registered claim anchor, a remaining-open-proposition anchor, an authored formal anchor without a registered claim, or a navigation anchor. Two consequences of this typing carry most of the value. Every remaining open proposition must resolve to exactly one authored problem environment across the paper suite: an open problem cannot be dropped from the exposition, or duplicated into two competing statements, without the gate failing. And the rendered PDFs carry page-level phrase contracts: named boundary sentences are required to appear on named early pages, so the load-bearing negative information provably survives into what a first-time reader actually sees.

7 Budgeted projections

Everything a reader or an agent meets before the papers is generated: the orientation documents, the corpus descriptor, the declaration atlas, the README’s scale and anchor regions, and the bounded query interface. Three rules govern this layer.

Builders own generated regions. Generated files and regions are marked, rebuilt by named scripts, and compared byte-for-byte in the gate; a hand edit to a generated surface is a release failure by construction. Content digests tie the layers together — the descriptor records SHA-256 digests of the paper sources, the rendered PDFs, and the alias crosswalk — so a stale artifact cannot silently represent a newer source.

First contact is budgeted. The README must fit in 12,000 bytes, the scope statement in 4,000, the human orientation document in 16,000, the machine orientation capsule in 32,000, the corpus descriptor in 64,000, and any entry route’s total first-contact bundle in 48,000; the bounded query tool caps each packet at 16,384 bytes and a summary at 32,000. Budgets are enforced in the gate, so surface growth becomes a visible editorial decision — compress or re-route — rather than silent accretion. Twice during development a budget failed and was met by removing duplication rather than by raising the number; both incidents are recorded in Section 9.2.

Entry routes must be bounded. Machine entry points are typed routes with intents, bounded reads, and query steps; a route is forbidden from sending first contact to an exhaustive owner such as the full registry or the declaration atlas. Programme-level routes must additionally state an explicit negative claim ceiling — the checker literally requires wording such as “remain open” or “does not” — so that even the navigation layer carries the open boundary.

8 The release gate

One command, [the release checker](#), runs the whole contract: 5,207 checks in 144.5 seconds at the checkpoint studied here, implemented in dependency-free Python over the working tree and the pinned Git history. The gate begins with the cheapest high-severity invariant: a proof-trust scan rejecting `sorry`, `admit`, project-defined axioms, and native evaluation in every Lean source, with comment- and string-aware lexing so prose mentions do not mask executable occurrences. It then verifies, in order: registry well-formedness and status typing; release identity across the build manifest, citation file, and toolchain pin; the existence and location of every claimed declaration at the pinned commit; every paper source link; scope and README boundary sentences, including a banned-phrase list and a leak check for internal machine identifiers; licens-

ing completeness; agent-entry routing; methodology contract validity and projection identity; module-graph, atlas, coordinate, descriptor, and alias freshness by rebuild-and-compare; the rendered-paper source contract; and the bounded-query and cold-clone comprehension suites.

The design decision that distinguishes the gate from a conventional test suite is that it validates its own detectors. Thirty-eight negative fixtures run inside the gate itself: nine methodology mutations (a conditional reduction without a remaining-open link, a finite result restated as open, a semantic change with downgraded review, and six more) must each fail contract validation; twenty-two adversarial mutations of the human first-contact surfaces and query packets must each be rejected by the cold-clone checker; five fixtures pin the proof-trust scanner against known-bad and known-good inputs; and two fixtures confirm the public-boundary test still rejects deleted provenance non-claims. A checker whose fixtures stop failing fails the release. This matters under maintenance: the easiest way to defeat a guard is not to attack it but to weaken it while editing something nearby, and the fixtures convert that class of regression into a visible failure.

9 Evaluation

9.1 Mutation study

The gate’s checks were exercised end-to-end by a mutation study at the release-green checkpoint: an isolated worktree was pinned at a commit where all 5,207 checks pass, and ten corruptions — one per failure class of Section 3, plus proof-trust and partition classes — were applied one at a time, running the full gate after each and restoring the tree between runs. Table 1 reports the outcome. Nine of the ten mutations were detected on the first gate run after they were applied, each by a check whose failure message names either the corrupted surface or the stale projection the corruption forces. Detection cost varied by two orders of magnitude: the proof-trust mutation was rejected after 4.7 seconds and 639 checks, before any expensive validation ran, while registry and surface mutations were caught by the full pass.

One mutation escaped. Weakening the finite-evidence boundary clause — replacing “does not supply the unbounded quantifier” by “completes the unbounded quantifier” in the README status table — passed every check at the studied checkpoint: the clause appeared in no anchor group, and nothing else inspects that sentence’s meaning. This is the most instructive outcome of the study, because it exhibits the enumeration boundary of Section 10 on a live surface rather than in principle: anchored sentences are protected, and the protection extends exactly as far as someone has thought to enumerate. The repair followed the architecture’s own idiom. The clause became a new anchor group in the headline-status task, from which the comprehension suite derives its rejection mutant automatically, so the escaped mutation is now a permanent negative fixture; the repaired checker rejects the same corruption at the same checkpoint.

Three cautions on reading the table. The mutations were written by the same author as the checks, so this is a coverage demonstration, not an independent red-team exercise. One mutation per class says nothing about subtler corruptions within a class — a status upgrade between adjacent rungs of the taxonomy that happens to satisfy every structural rule would not be caught by structure alone, which is why status-changing edits are also bound to mandatory human review by the methodology contract. And detection is reported at the level of the failing check family; several mutations additionally triggered secondary failures downstream, which is the intended behaviour of a digest-chained system.

9.2 Defects caught during development

Three incidents from this corpus’s own history, all public in the repository record, show the same checks firing on real defects rather than planted ones.

Mutation (one deliberate corruption)	Detected by
Claim status upgraded: <i>conditional reduction</i> → <i>proved here</i>	forced projection staleness (descriptor rebuild drift)
README plain-language boundary removed (“does not solve”)	README plain-language boundary check
Declaration coordinate drifted by +40 lines	source-coordinate freshness check; descriptor drift
Paper source link retargeted (+50 lines)	source-coordinate freshness check; paper source digest
One claim placed in two contribution families	partition disjointness check, naming the claim
Generated descriptor hand-edited (module count +1)	descriptor rebuild drift; module-count cross-check
<code>native_decide</code> introduced into a Lean proof	proof-trust scan, fail-fast in 4.7 s
Finite-evidence clause weakened (“does not supply” → “completes”)	<i>escaped</i> : clause was in no anchor group; see text
Authored open-problem anchor title mangled	anchor typing; open-proposition-to-anchor resolution
Orientation surface padded past its byte budget	route first-contact bundle budget; descriptor drift

Table 1: Ten mutations at a checkpoint where all 5,207 checks pass. Each was applied alone; the full gate was run after each; nine were detected, and the escape (row 8) is analysed in the text. The right column names the first failing check family observed.

First, boundary erosion: during a July 2026 editing pass, a wording improvement to the README dropped the negative clause stating that an exact final-skip band formula “does not show that the actual orbit avoids” an unsafe band — exactly the clause distinguishing a checked identity from the open avoidance statement. The cold-clone comprehension check failed the gate within the same working session, and the sentence was restored before the edit could land.

Second, attention overflow: the human orientation document reached 16,553 bytes against a 16,000-byte budget after new claim records landed, and the bounded query summary reached 33,393 bytes against 32,000. Both overflows blocked the release gate and were resolved by removing duplicated route text — title-restating intent lines, per-route command strings derivable from route identifiers — rather than by raising either budget.

Third, coordinate rot at small scale: a five-line metadata addition to a paper preamble shifted every subsequent line coordinate in that file; the coordinate refresh and digest chain surfaced the stale projections immediately, so the regenerated surfaces land with the edit rather than after it. Each incident is small; that is the point. None of the three would have announced itself, and two of the three were introduced by tooling-assisted edits whose diffs looked entirely plausible in review.

9.3 Cost

The complete machinery is 21 Python files totalling roughly 7,800 lines, using only the standard library, with no network access and no service dependencies; the registry, methodology contract, and generated surfaces are plain JSON and Markdown in the repository. The full gate runs in about 145 seconds on commodity hardware, fast enough to run before every landing. Maintaining the contract costs one structured edit per claim — a status, a declaration coordinate, a family assignment, and any open-obligation link — which for this corpus has been absorbed into the same editing passes that produce the claims.

10 What this architecture does not establish

This is a case study of one corpus, one maintainer, and one working style; no controlled comparison against conventional practice was attempted, and a disciplined author with ordinary continuous integration might avoid many of the same defects at this scale. The narrower claim defended here is that the publication-faithfulness invariants of Sections 4–8 are checkable at all, at commodity cost, and that they fired on real defects.

The gate checks presence and consistency, not meaning. Anchored sentences are protected; unanchored prose can still be mathematically inadequate in ways no structural check will notice, which is why the methodology contract refuses to let any semantic change class bypass human mathematical review. The status taxonomy, anchor sentences, budgets, and non-claims are hand-designed for this corpus; the design transfers as a pattern, but nothing here automates the judgement of which sentences are load-bearing. Detection is not prevention: a maintainer can still weaken a check, and the in-gate fixtures raise the cost of doing so accidentally without making it impossible. Finally, the proof authority is inherited entirely from Lean and Mathlib; nothing in this paper strengthens it, and kernel acceptance itself does not certify that a formal statement captures its intended informal meaning — that residue of interpretation is exactly why the claim registry exists as a human-reviewed object rather than a generated one.

11 Related work

The Mathlib community paper [4] describes the architecture and social organisation of a large general-purpose library, and *Growing Mathlib* [5] analyses its maintenance: deprecation machinery, linters, build performance, technical debt, and review tooling. Both concern the health of the formal library itself. The present work is orthogonal: it takes the formal layer’s health as given and governs the faithfulness of everything published *about* it.

The closest relatives are formalisation blueprints. Massot’s `leanblueprint` [6] maintains a human-readable account of a proof linked to its Lean formalisation, with a dependency graph and per-node formalisation status; it has organised flagship developments including the PFR formalisation [8] and the Carleson project [9]. LeanArchitect [7] inverts the authoring direction, extracting blueprint structure and status directly from annotated Lean source and synchronising the $\text{\LaTeX}$ automatically. Blueprints and this architecture answer different questions at different times. A blueprint tracks *coverage* during construction: which nodes of an intended proof are formalised. The machinery here governs *claim strength* at publication: which logical status each public sentence may assert, whether the negative boundaries survive editing, and whether every projection agrees — concerns that remain after coverage tracking ends, and that apply to corpora (like this one) whose primary outputs include exact reductions and open boundaries rather than a single target theorem. The two compose naturally, and nothing here replaces a blueprint.

The Archive of Formal Proofs [10] demonstrates long-lived refereed distribution of formal developments with sustained build guarantees; its gates concern build health and editorial acceptance rather than the status faithfulness of prose surfaces. We are not aware of prior work that gates a release on machine-checked agreement between a claim registry, authored exposition, generated navigation, and pinned formal sources; we would be glad to learn of antecedents.

12 Reusing the pattern

Stated as commitments rather than code, the architecture transfers to any project whose public surfaces restate machine-checked content:

1. Record the authority order as data, and require every evidence class and every layer to state what it does not decide.
2. Make claim status a typed object: a closed taxonomy with structural rules, first-class open obligations with a closed effect vocabulary, and explicit non-claims — including provenance non-claims if the public artifact projects a larger private effort.
3. Let authored prose carry verifiable coordinates inside readable words; pin links to immutable revisions; forbid floating references; and require every open obligation to own exactly one authored statement.
4. Generate every navigation surface from the registry, chain the layers with content digests, and put first-contact surfaces under byte budgets enforced at release.
5. Gate releases on all of the above with one cheap command, and give every checker negative fixtures that run inside the gate, so the detectors are themselves under test.

What does not transfer for free: the taxonomy, the anchor sentences, the budgets, and the family partition are editorial judgements that must be re-authored per corpus, and the implementation here is project-local script rather than a reusable package. Porting the pattern is a re-implementation on the order of a few thousand lines, not an installation.

13 Complements and further questions

Four questions remain open on the engineering side. Whether any useful fraction of *unanchored* prose adequacy can be checked — beyond presence of named sentences — without drifting into checking style rather than truth. Whether the registry and contract vocabularies can be shared across corpora without flattening exactly the distinctions that make a per-corpus taxonomy honest. Whether parts of the gate belong inside the proof assistant itself, in the direction LeanArchitect takes for blueprints, rather than beside it in repository tooling. And how the balance shifts as more of the editing pressure comes from AI assistance: the incidents of Section 9.2 suggest that as prose generation gets cheaper, release-time faithfulness checking stops being optional hygiene and becomes the load-bearing control, but one project’s history is evidence only that the failure mode is real, not that this is the right general control. The underlying mathematical corpus, meanwhile, keeps its own open boundary: Erdős problems 249 and 257 remain open, and every surface this paper described exists to keep saying so exactly.

References

- [1] W. Cook, *Tail Certificates and Achievement-Set Geometry for Erdős Problems 249 and 257*, gateway exposition in this release.
- [2] W. Cook, *Finite-Difference Certificates and a Phase-Separation Characterisation for Erdős Problem 249*, companion note in this release.
- [3] L. de Moura, S. Kong, J. Avigad, F. van Doorn, and J. von Raumer, *The Lean theorem prover*, in *Automated Deduction — CADE-25*, Lecture Notes in Computer Science 9195, Springer, 2015, pp. 378–388.
- [4] The mathlib Community, *The Lean mathematical library*, in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)*, ACM, 2020, pp. 367–381, doi:[10.1145/3372885.3373824](https://doi.org/10.1145/3372885.3373824).

- [5] A. Baanen et al., *Growing Mathlib: maintenance of a large scale mathematical library*, in *Intelligent Computer Mathematics (CICM 2025)*, Lecture Notes in Computer Science, Springer, 2026, [arXiv:2508.21593](#).
- [6] P. Massot, *leanblueprint: a plasTeX plugin for formalization blueprints*, [github.com/PatrickMassot/leanblueprint](#), accessed July 2026.
- [7] T. Zhu, P. Monticone, J. Avigad, and S. Welleck, *LeanArchitect: Automating Blueprint Generation for Humans and AI*, 2026, [arXiv:2601.22554](#).
- [8] T. Tao, *Formalizing the proof of PFR in Lean4 using Blueprint: a short tour*, blog post, November 2023, [terrytao.wordpress.com](#).
- [9] L. Becker et al., *A blueprint for the formalization of Carleson’s theorem on convergence of Fourier series*, 2024, [arXiv:2405.06423](#).
- [10] *The Archive of Formal Proofs*, [isa-afp.org](#), accessed July 2026.